

CLASSICAL MACHINE LEARNING

Which Model When

A complete field reference for model selection, evaluation, tuning, and shipping. Built for interview revision and real project decisions.

MODEL SELECTION · DECISION TREES · SCIKIT-LEARN SNIPPETS
METRICS · HYPERPARAMETER TUNING · IMBALANCED DATA
FEATURE ENGINEERING · END-TO-END PIPELINE · INTERVIEW Q&A

Inside this reference

01	What is tabular data + how to read this	p.3
02	The master decision flow	p.3
03	"Which algorithm should I use?" decision trees	p.4
04	Supervised models: cards, code, interview Q	p.5
05	Unsupervised models: clustering & PCA	p.9
06	Scaling & normalization	p.11
07	Evaluation metrics (with visuals)	p.12
08	Bias-variance & over/underfitting	p.14
09	Hyperparameter tuning cheat sheet	p.15
10	Imbalanced datasets: SMOTE & class weights	p.16
11	Feature engineering & preprocessing checklist	p.17
12	End-to-end ML pipeline: raw data to deployment	p.18
13	Last-minute revision sheet	p.20

How to use it. Sections 01-03 get you to a shortlist fast. Sections 04-05 are the model deep-dives with copy-paste scikit-learn and the interview question you will actually be asked. Sections 06-12 are the surrounding craft that separates a working model from a shipped one. Section 13 is the page to read the night before.

01 What is tabular data?

Rows are records, columns are features. A spreadsheet. Almost all classical ML lives here. This one loan table is reused across every model in this reference, so you can see exactly what each algorithm does to the same data.

AGE	INCOME	LOAN AMT	CREDIT SCORE	EMPLOYED	APPROVED (TARGET)
28	45,000	15,000	680	Yes	Yes
45	90,000	50,000	720	Yes	Yes
33	30,000	40,000	590	No	No
52	120,000	20,000	810	Yes	Yes

Each **row** is one applicant. Each **column** is a feature. The last column, **Approved**, is the **target** we want to predict. The three ingredients that drive every model choice are hiding in this table: is the target a **label** or a **number** or **absent**, how many **rows** you have, and how many **columns**.

02 The master decision flow

- 1 What is the output?** A category to predict → classification. A number → regression. No target at all → clustering or dimensionality reduction. This is the only hard fork.
- 2 Must you explain every prediction?** If a stakeholder needs to trust and question each call, interpretability wins: start with Logistic/Linear Regression or a single Decision Tree.
- 3 Want best accuracy on tabular data?** Go straight to Gradient Boosting (XGBoost / LightGBM). Want strong results with near-zero tuning? Random Forest.
- 4 Small or high-dimensional data?** SVM or a regularized linear model. Sparse text? Naive Bayes or linear SVM.
- 5 Always fit a baseline first.** Even when you are sure you will use boosting. It tells you whether the problem is easy and gives every fancier model a number to beat.

The choice is empirical, not a lookup. "No free lunch" says no model wins on every dataset. You shortlist two or three with the flow above, then cross-validate and let the metric decide. And pick the metric **before** you compare.

03 "Which algorithm should I use?"

Three decision trees for the three situations you will actually face. Follow the branches to a shortlist.

TREE A PREDICTING A CATEGORY (CLASSIFICATION)

START: target is a label (Yes/No, spam/ham, class A/B/C)

- |
- |— Need a human-readable rule for compliance? —> **Decision Tree** / Logistic Regression
- |
- |— Is it text / very sparse high-dim features? —> **Naive Bayes** or Linear SVM
- |
- |— Small data (< ~10k rows), many features? —> **SVM** (RBF kernel)
- |
- |— Want strong result, almost no tuning? —> **Random Forest**
- |
- |— Want the best possible accuracy on tabular? —> **XGBoost / LightGBM**
(baseline with Logistic first)

TREE B PREDICTING A NUMBER (REGRESSION)

START: target is continuous (price, amount, temperature)

- |
- |— Relationship looks linear + want coefficients? —> **Linear Regression**
- |
- |— Many features / overfitting risk? —> **Ridge / Lasso / ElasticNet**
(Lasso also selects features)
- |
- |— Non-linear, tabular, want best accuracy? —> **Gradient Boosting Regressor**
- |
- |— Non-linear but want low tuning? —> **Random Forest Regressor**

TREE C NO TARGET AT ALL (UNSUPERVISED)

START: no label column, looking for structure

- |
- |— Want to GROUP records?
 - |— Know cluster count, compact blobs? —> **K-Means**
 - |— Odd shapes + want outliers flagged? —> **DBSCAN**
 - |— Want a hierarchy / dendrogram? —> **Agglomerative**
- |
- |— Too many columns to model?
 - |— Compress while keeping variance —> **PCA**

Cross-cutting rule. When two candidates look equally good on paper, prefer the simpler and more interpretable one, then let cross-validated performance break the tie. Complexity has to earn its place against the baseline.

04 Supervised models

Each card: when to use and skip, what to watch, the same loan-table example, copy-paste scikit-learn, and the interview question you should expect.

Linear & Logistic Regression

BASELINE · INTERPRETABLE · FAST | Linear reg = number out · Logistic = probability out

Use Linear relationships, need explainable coefficients, want a baseline before anything heavier.

Skip Strong non-linearities or heavy feature interactions no engineering fixes.

Watch Scale features, check multicollinearity, add L1/L2 when features outnumber clean signal.

Example Predict **Approved** (logistic) with a coefficient per feature; or predict **Loan Amount offered** (linear).

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

model = make_pipeline(StandardScaler(),
                      LogisticRegression(C=1.0, max_iter=1000))
model.fit(X_train, y_train)
proba = model.predict_proba(X_test)[:, 1] # probability of approval
```

Interview: Why is it called logistic *regression* if it does classification? Because it regresses the log-odds (a linear function) and passes it through the sigmoid to get a probability, then thresholds it.

Decision Tree

INTERPRETABLE · NON-LINEAR · UNSTABLE · NO SCALING NEEDED

Use Need a readable rule set, mixed feature types, non-linear splits.

Skip You need stable, top accuracy. One tree overfits easily.

Watch Cap depth or prune. Tiny data changes reshape the whole tree.

Example Outputs a printable rule: **if credit score < 600 and employed = No then reject.**

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

clf = DecisionTreeClassifier(max_depth=4, min_samples_leaf=20,
                           criterion="gini")
clf.fit(X_train, y_train)
plot_tree(clf, feature_names=cols, filled=True) # visualize rules
```

Interview: Gini vs entropy? Both measure node impurity and usually give near-identical trees. Gini is slightly faster (no log); entropy can be marginally more balanced. The split criterion matters far less than depth control.

Random Forest

ROBUST · LOW-TUNING · BAGGING · STRONG TABULAR DEFAULT

Use Best no-effort default for tabular. Handles non-linearity and noise, gives feature importance.

Skip Very large data where boosting wins, or when you need the single best score.

Watch Bigger and slower than boosting at similar accuracy; less interpretable than one tree.

Example Hundreds of trees on bootstrapped samples vote on **Approved**. Far steadier than one tree.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=300, max_depth=None,
                           max_features="sqrt", n_jobs=-1,
                           class_weight="balanced")
rf.fit(X_train, y_train)
importances = rf.feature_importances_
```

Interview: Why does a forest beat a single tree? Bagging: each tree trains on a bootstrap sample and a random feature subset, so their errors are decorrelated. Averaging many high-variance trees cancels the variance without adding bias.

Gradient Boosting (XGBoost / LightGBM)

TOP TABULAR ACCURACY · SEQUENTIAL · TUNABLE

Use The go-to for winning tabular performance, competitions, most production tabular tasks.

Skip Tiny datasets, or when full interpretability is mandatory.

Watch Sensitive to hyperparameters; use early stopping and a small learning rate to avoid overfit.

Example Same loan table at **200k rows, 40 columns**. Squeezes out the highest accuracy.

```
from xgboost import XGBClassifier

xgb = XGBClassifier(n_estimators=600, learning_rate=0.05,
                   max_depth=5, subsample=0.8,
                   colsample_bytree=0.8, eval_metric="aucpr",
                   scale_pos_weight=ratio) # for imbalance
xgb.fit(X_train, y_train,
        eval_set=[(X_val, y_val)], verbose=False) # early stop
```

Interview: Bagging vs boosting? Bagging (RF) builds trees in *parallel* to cut variance. Boosting builds them *sequentially*, each correcting the previous one's errors, to cut bias. Boosting usually scores higher but is more prone to overfitting.

Support Vector Machine (SVM)

SMALL DATA · HIGH-DIMENSIONAL · MARGIN-BASED · MUST SCALE

Use Small-to-medium data, high-dim spaces (text, genomics), clear margins. Kernels for non-linear.

Skip Large datasets (slow), or when you need cheap probability outputs.

Watch Scale features. Kernel, C, and gamma tuning matter a lot.

Example **80 applicants, 300 columns.** Few rows, many features: SVM handles that shape.

```
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

svm = make_pipeline(StandardScaler(),
                    SVC(kernel="rbf", C=1.0, gamma="scale", probability=True))
svm.fit(X_train, y_train)
```

Interview: What is the kernel trick? It computes dot products in a high-dimensional space without ever mapping the data there explicitly, letting a linear margin separate data that is non-linear in the original space. RBF is the common default.

k-Nearest Neighbors (kNN)

NO TRAINING · LAZY · DISTANCE-BASED · MUST SCALE

Use Small datasets, low dimensions, quick baseline, similarity-style recommendation.

Skip Large or high-dim data. Prediction is slow and distances lose meaning.

Watch Scale features, choose k carefully; suffers the curse of dimensionality.

Example New applicant arrives → find the **5 most similar** past applicants, predict their majority outcome.

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5, weights="distance")
knn.fit(X_train_scaled, y_train)
```

Interview: Is kNN parametric? No, it is non-parametric and lazy: there is no training phase and no learned parameters. All work happens at prediction time by comparing to stored points.

Naive Bayes

TEXT · FAST · PROBABILISTIC · SPARSE-FRIENDLY

Use Text classification, spam filtering, high-dim sparse features, ultra-fast baseline.

Skip Strongly correlated features, or when you need calibrated probabilities.

Watch The independence assumption is usually false but works surprisingly often.

Example Table of emails, columns are **word counts** → predict spam / not spam.

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import TfidfVectorizer

pipe = make_pipeline(TfidfVectorizer(), MultinomialNB(alpha=1.0))
pipe.fit(texts_train, y_train)
```

Interview: Why "naive"? It assumes all features are conditionally independent given the class, which is naive but makes the math trivial and fast, and the ranking of classes often stays correct even when the assumption is violated.

05 Unsupervised models

No target column. You are finding structure, not predicting a label.

K-Means

CLUSTERING · FAST · ASSUMES SPHERICAL · MUST SCALE

Use You roughly know the number of clusters; groups are compact and similar in size.

Skip Irregular shapes, varying density, or unknown k.

Example Drop **Approved**. Group customers by age and income into 3 segments.

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, n_init=10, random_state=42)
labels = km.fit_predict(X_scaled)
```

Interview: How do you choose k? The elbow method (plot inertia vs k, pick the bend) or the silhouette score. Never optimize inertia alone: it always falls as k grows.

DBSCAN

DENSITY · FINDS OUTLIERS · NO k

Arbitrary shapes, unknown count, automatic outlier flagging. Struggles with clusters of very different densities. Tune **eps** and **min_samples**.

```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(X_scaled)
# label -1 = noise/outlier
```

Hierarchical

DENDROGRAM · NESTED GROUPS

Want a cluster hierarchy without presetting k. Scales poorly on large data. Linkage choice (ward, average, complete) changes results.

```
from sklearn.cluster import \
    AgglomerativeClustering
ac = AgglomerativeClustering(
    n_clusters=3, linkage="ward")
labels = ac.fit_predict(X_scaled)
```

PCA (dimensionality reduction)

LINEAR · PREPROCESSING · MUST SCALE FIRST

Use Reduce dimensions, remove correlation, speed up models, visualize variance structure.

Skip You need interpretable features, or structure is non-linear (use t-SNE / UMAP to visualize only).

Example 50 correlated financial columns compressed into ~5 combined dimensions.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=0.95) # keep 95% of variance
X_reduced = pca.fit_transform(X_scaled)
print(pca.explained_variance_ratio_.sum())
```

Interview: What do principal components represent? Orthogonal directions of maximum variance in the data. PC1 captures the most variance, PC2 the most of what remains, and so on. Components are combinations of original features, not the features themselves.

06 Scaling & normalization

Scaling matters when a model measures **distance** or uses **gradients / regularization**. It is irrelevant to tree-based models, which split one feature at a time. Always fit the scaler on the **training fold only**.

MODEL	SCALE?	WHY
SVM	Yes	Uses distances and margins
kNN	Yes	Pure distance; large-range features dominate
K-Means	Yes	Distance to centroids
PCA	Yes	Variance-driven; scale sets the axes
Linear / Logistic + L1 or L2	Yes	Penalty is unfair across different scales
Decision Tree	No	Splits one feature at a time
Random Forest	No	Tree-based, scale-invariant
Gradient Boosting	No	Tree-based, scale-invariant
Naive Bayes	No	Works on probabilities / counts

Standardization (Z-score)

MEAN 0 · STD 1 · SAFE DEFAULT

For roughly bell-shaped data, and for SVM, linear models, PCA.

```
from sklearn.preprocessing import \
    StandardScaler
Xs = StandardScaler().fit_transform(X)
```

Normalization (Min-Max)

RANGE 0..1 · BOUNDED

When you need a fixed range or non-Gaussian data.
Sensitive to outliers.

```
from sklearn.preprocessing import \
    MinMaxScaler
Xn = MinMaxScaler().fit_transform(X)
```

Outliers present? Use **RobustScaler** (centers on the median, scales by IQR). And never fit any scaler on the full dataset before splitting: that leaks test information into training.

07 Evaluation metrics

Pick the metric **before** comparing models. The wrong metric silently selects the wrong model, especially on imbalanced data.

		Predicted	
		Positive	Negative
Actual	+	TP actual + / pred +	FN actual + / pred -
	-	FP actual - / pred +	TN actual - / pred -

Precision = $TP / (TP + FP)$ of predicted +, how many were right

Recall = $TP / (TP + FN)$ of actual +, how many were caught

The confusion matrix is the source of every classification metric.

PRECISION VS RECALL

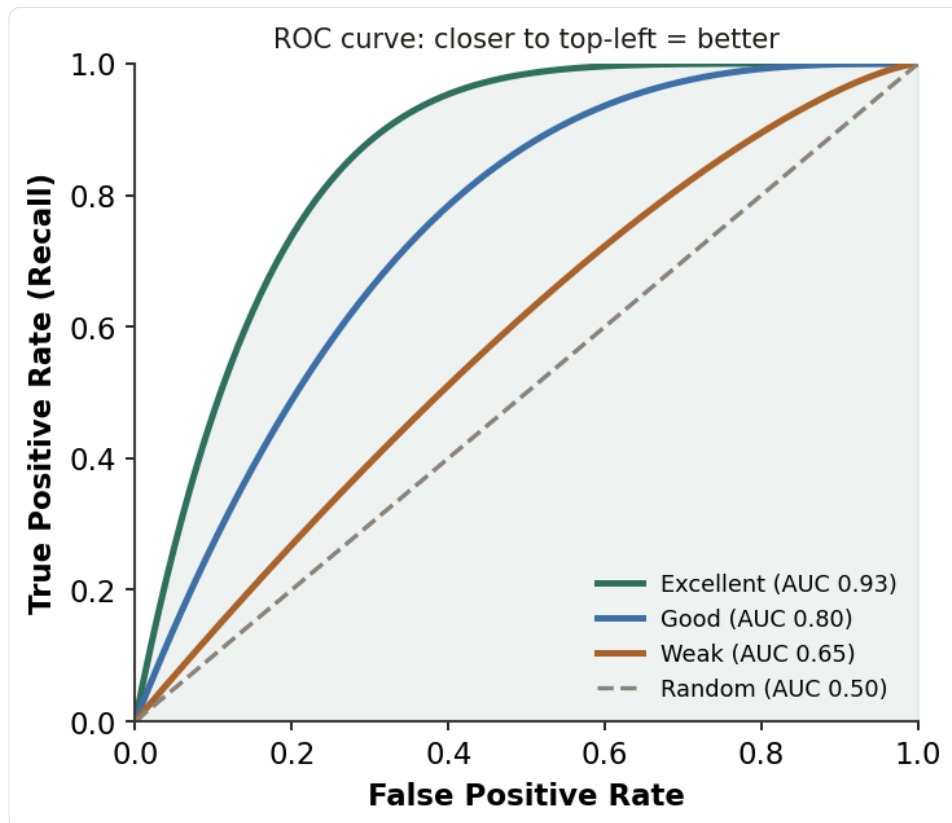
Precision asks: of everything I flagged positive, how much was right? Optimize it when **false positives are costly** (flagging a good customer as fraud).

Recall asks: of all real positives, how many did I catch? Optimize it when **false negatives are costly** (missing a fraud or a disease).

F1 is their harmonic mean: use it when you need both and classes are imbalanced.

CLASSIFICATION METRICS

METRIC	USE WHEN	CAREFUL
Accuracy	Classes are balanced	Lies on imbalance (99% "no fraud")
Precision	False positives costly	Ignores missed positives
Recall	False negatives costly	Ignores false alarms
F1	Need precision + recall balance	Hides which one is weak
ROC-AUC	Ranking quality, roughly balanced	Over-optimistic on heavy imbalance
PR-AUC	Imbalanced, positive class matters	The right AUC for rare events



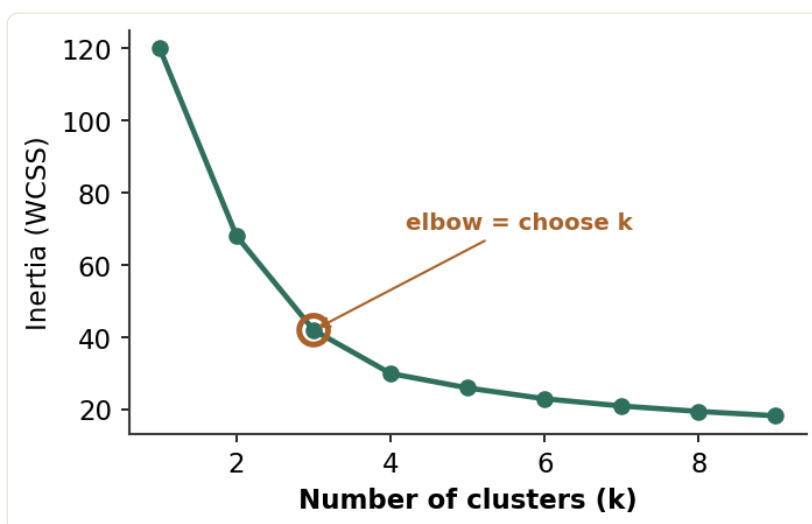
ROC-AUC: probability the model ranks a random positive above a random negative. 0.5 = random, 1.0 = perfect.

REGRESSION METRICS

METRIC	USE WHEN	CAREFUL
MAE	Want error in real units, robust to outliers	Treats all errors linearly
MSE / RMSE	Large errors should hurt more	Outliers dominate
R ²	How much variance is explained	Rises just by adding features (use adjusted R ²)
MAPE	Want error as a percentage	Breaks near zero values

CLUSTERING METRICS

METRIC	USE WHEN	CAREFUL
Silhouette	No true labels; measures separation	Assumes convex clusters
Inertia / Elbow	Choosing k for K-Means	Always drops; judge the bend
Adjusted Rand	You have ground-truth labels	Needs true labels



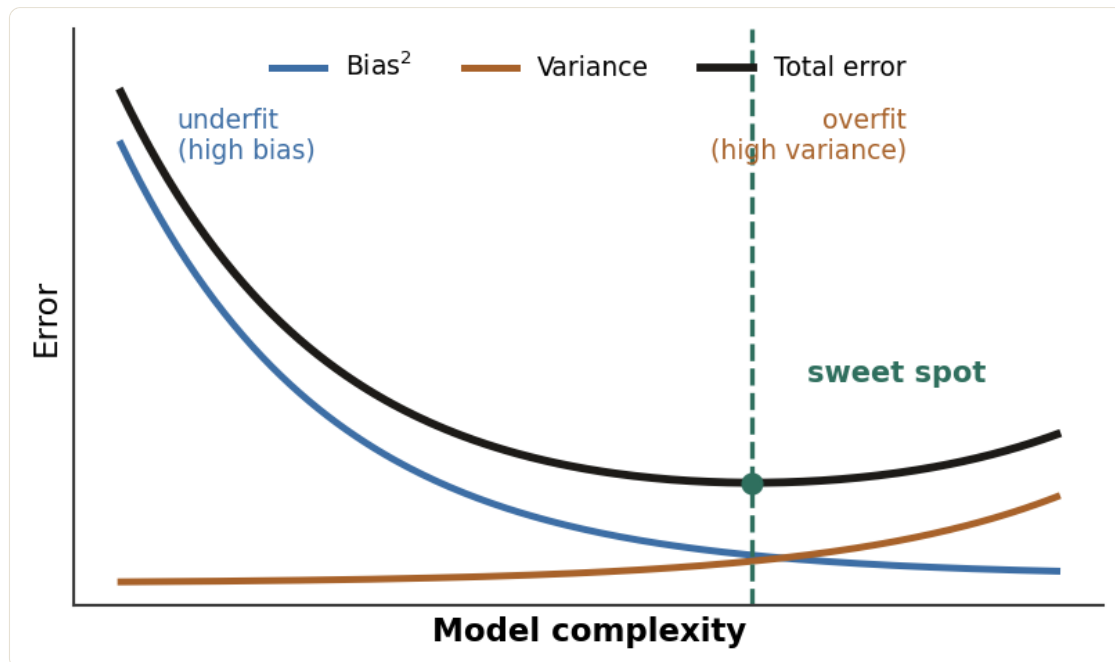
Elbow method for K-Means: pick k at the bend where extra clusters stop paying off.

Fraud example. 99.5% of transactions are legit. A model predicting "all legit" scores 99.5% accuracy and catches zero fraud. Judge it with **recall** and **PR-AUC** instead.

```
from sklearn.metrics import (classification_report,
                             roc_auc_score, average_precision_score)
print(classification_report(y_test, preds))
print("PR-AUC:", average_precision_score(y_test, proba))
```

08 Bias, variance & fit

Every model error decomposes into bias, variance, and irreducible noise. Managing the first two is most of the job.



Total error is U-shaped in complexity. Too simple = high bias (underfit). Too complex = high variance (overfit).

High bias (underfit)

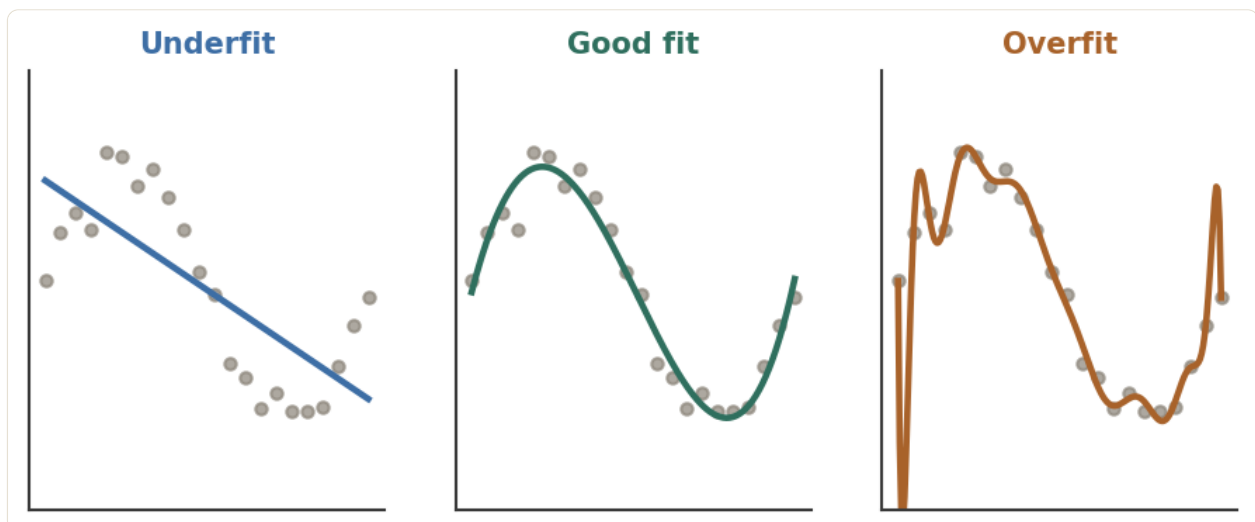
TOO SIMPLE

Poor on train *and* test. Model can't capture the pattern. Fix: more complex model, more features, less regularization.

High variance (overfit)

TOO COMPLEX

Great on train, poor on test. Model memorized noise. Fix: more data, regularization, simpler model, bagging.



The same data fit three ways: underfit misses the shape, good fit tracks the signal, overfit chases every point.

Where models sit. Linear/logistic = high bias, low variance. Deep trees and low-k kNN = low bias, high variance. Bagging (Random Forest) attacks variance; boosting attacks bias. The bias-variance tradeoff is why ensembles work.

09 Hyperparameter tuning

Defaults get you 80%. Tuning the few parameters that matter gets the rest. Tune inside cross-validation, never against the test set.

MODEL	PARAMETERS THAT MATTER MOST	DIRECTION
Logistic / Linear	C (inverse reg), penalty (l1/l2)	Lower C = more regularization
Decision Tree	max_depth, min_samples_leaf	Lower depth = less overfit
Random Forest	n_estimators, max_depth, max_features	More trees safe; cap depth to control
XGBoost / LightGBM	learning_rate, n_estimators, max_depth, subsample, colsample	Low LR + more trees + early stop
SVM	C, gamma, kernel	High C = tighter fit; tune gamma with C
kNN	n_neighbors, weights, metric	Higher k = smoother boundary
K-Means	n_clusters, n_init	Choose k via elbow / silhouette

SEARCH STRATEGIES

METHOD	WHEN	COST
GridSearchCV	Few params, small ranges	Exhaustive, expensive
RandomizedSearchCV	Many params, limited budget	Efficient, usually preferred
Bayesian (Optuna)	Expensive models, big spaces	Smartest, needs a library

```

from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

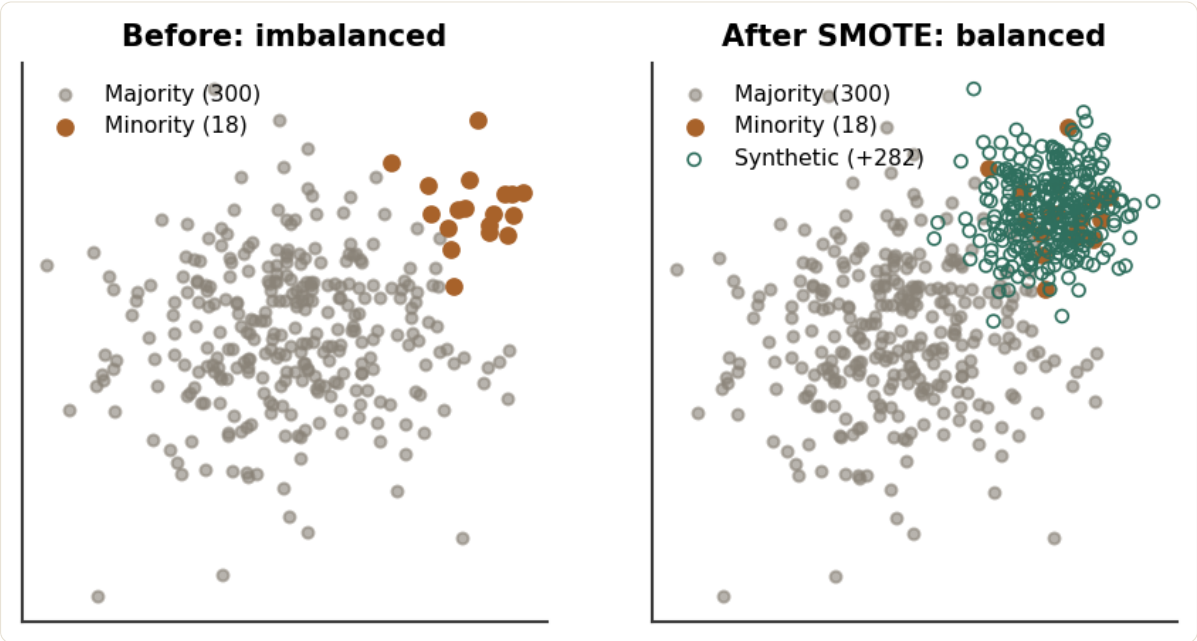
params = {"max_depth": randint(3, 10),
          "learning_rate": uniform(0.01, 0.2),
          "subsample": uniform(0.6, 0.4)}
search = RandomizedSearchCV(XGBClassifier(), params, n_iter=40,
                           scoring="average_precision", cv=5, n_jobs=-1)
search.fit(X_train, y_train)
print(search.best_params_)

```

Order of operations. Fix a good scoring metric first, use a small learning rate with early stopping for boosting, tune the high-impact params before the rest, and always report the cross-validated score, not the training score.

10 Imbalanced datasets

Fraud, churn, disease, defaults: the interesting class is rare. Accuracy is useless here. Three levers: resample, reweight, and re-metric.



SMOTE creates synthetic minority points by interpolating between real ones, balancing the classes without naive duplication.

TECHNIQUE	WHAT IT DOES	WATCH
Class weights	Penalize minority errors more during training	First thing to try; zero data change
SMOTE	Synthesize new minority samples by interpolation	Fit on train only, inside CV
Random oversampling	Duplicate minority rows	Can overfit to copies
Random undersampling	Drop majority rows	Throws away data
Threshold tuning	Move the decision cutoff off 0.5	Cheap, often overlooked

CLASS WEIGHTS (START HERE)

```
# Most sklearn models accept this directly
LogisticRegression(class_weight="balanced")
RandomForestClassifier(class_weight="balanced")
SVC(class_weight="balanced")
# XGBoost uses a ratio instead
XGBClassifier(scale_pos_weight=neg_count/pos_count)
```

SMOTE (INSIDE A PIPELINE SO IT NEVER TOUCHES TEST DATA)

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline

pipe = Pipeline([
    ("smote", SMOTE(random_state=42)),
    ("clf", RandomForestClassifier(n_estimators=300))]
)
pipe.fit(X_train, y_train)  # SMOTE runs on train folds only
```

The cardinal sin: applying SMOTE before the train/test split. Synthetic points leak into the test set and inflate your score. Always resample inside the CV pipeline. And judge with **recall / F1 / PR-AUC**, never accuracy.

11 Feature engineering & preprocessing checklist

Better features beat better models more often than people admit. Work through this before reaching for a fancier algorithm.

CLEANING

STEP	HOW
Missing values	Impute (median for skewed, mean for symmetric, mode for categorical); add a "was-missing" flag
Outliers	Cap/winsorize, or use RobustScaler; investigate before deleting
Duplicates	Drop exact dupes; check near-dupes
Data types	Parse dates, cast numerics, strip whitespace

ENCODING CATEGORICALS

METHOD	USE WHEN
One-hot	Low cardinality, no order (color, city)
Ordinal	Natural order (small/medium/large)
Target / mean encoding	High cardinality; guard against leakage with CV folds
Frequency encoding	High cardinality, quick baseline

CREATING & SELECTING

TECHNIQUE	WHAT IT BUYS YOU
Interaction / ratio features	Loan-to-income ratio often beats either column alone
Binning	Turn age into bands when the effect is non-linear
Date parts	Day-of-week, month, is-weekend from a timestamp
Log transform	Tame skewed money/count features
Feature selection	Lasso, tree importance, or mutual information to cut noise

Leakage is the number-one silent killer. Fit every transformer (scaler, encoder, imputer, SMOTE) on the training fold only. Never use future information, the target, or test-set statistics to build a feature. Wrap the whole thing in a scikit-learn Pipeline so this is enforced automatically.

12 End-to-end ML pipeline

From raw table to a model serving predictions. The modeling is one box among many.

- 1 **Frame the problem.** Classification / regression / clustering? Define the target and the metric that maps to business value before touching data.
- 2 **Collect & split.** Assemble the data, then split train/validation/test *first*. Everything after this touches train only.
- 3 **EDA.** Distributions, correlations, missingness, class balance, leakage checks. Understand before modeling.
- 4 **Preprocess & engineer.** Impute, encode, scale, build features. All inside a Pipeline fit on train.
- 5 **Baseline.** Fit a simple model (logistic / linear). Every later model must beat this number.
- 6 **Model & tune.** Shortlist via the decision trees, cross-validate, tune the high-impact hyperparameters.
- 7 **Evaluate on held-out test.** Touch the test set once, with the metric chosen in step 1. Check for over/underfitting.
- 8 **Deploy.** Serialize the whole pipeline, wrap in an API, version it.
- 9 **Monitor.** Track data drift and metric decay; retrain on a schedule or trigger.

PERSIST THE WHOLE PIPELINE, NOT JUST THE MODEL

```
import joblib
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

pre = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)])

pipe = Pipeline([("pre", pre),
                  ("clf", XGBClassifier())])
pipe.fit(X_train, y_train)

joblib.dump(pipe, "model.joblib") # preprocessing travels with it
loaded = joblib.load("model.joblib")
loaded.predict(new_raw_rows)      # raw in, prediction out
```

Why bundle preprocessing? If you save only the model, production has to re-implement every scaling and encoding step by hand, and any mismatch silently corrupts predictions. Serializing the full Pipeline guarantees training and serving transform data identically.

13 Last-minute revision sheet

PICK A MODEL IN 10 SECONDS

Best tabular accuracy	XGBoost / LightGBM
No-tuning strong default	Random Forest
Explain to stakeholders	Logistic / single Tree
Text classification	Naive Bayes / Linear SVM
Small, high-dim data	SVM
Overfitting, many features	Lasso / ElasticNet
Group, know k	K-Means
Group, odd shapes	DBSCAN
Too many columns	PCA

MUST-KNOW ONE-LINERS

Bagging	Parallel trees, cuts variance (RF)
Boosting	Sequential trees, cuts bias (XGB)
High bias	Underfit: bad on train + test
High variance	Overfit: great train, bad test
Scale for	SVM, kNN, K-Means, PCA, reg. linear
Don't scale	Any tree-based model
L1 (Lasso)	Zeros weights, selects features
L2 (Ridge)	Shrinks smoothly, keeps all
Imbalance	Class weights / SMOTE + PR-AUC

METRIC PICKER

Balanced classes	Accuracy	False neg costly	Recall
False pos costly	Precision	Rare positive class	PR-AUC
Need both P & R	F1	Regression, punish big errors	RMSE

THE FIVE RULES THAT PREVENT 90% OF MISTAKES

- 1 Split before you do anything else. Fit transformers on train only.
- 2 Baseline first. Complexity must beat it to earn its place.
- 3 Pick the metric before comparing models.
- 4 Cross-validate. One split can lie.
- 5 Never let accuracy judge an imbalanced problem.

Classical ML Reference · v2 · No free lunch: shortlist with judgement, then let cross-validated evidence decide. Built for interview revision and real project decisions.